ECOMMERCE MANAGEMENT SYSTEM

USING SQL

Submitted By: Ajinkya Jayvant Bane

Guided By: Sameer Warsolkar

Abstract

This E-Commerce Management System is designed to organize the everyday information handled by an online shopping business. Instead of keeping customer, product, order, payment and delivery information separately in an unstructured way, the project connects these details through a relational database.

The goal is simple: store reliable data and make it easy to answer practical business questions. The project uses six related tables — Customers, Categories, Products, Orders, Payments and Delivery. Primary keys identify records and foreign keys connect related records. SQL queries are then used to explore sales, customers, products, payments and deliveries.

Tables Used

Table	Purpose
Customers	Stores customer details and acts as the main customer table.
Categories	Stores product category information.
Products	Stores products, prices, stock and category relationship.
Orders	Stores customer, purchases, quantities, dates and order amounts.
Payments	Stores payment details connected to orders.
Delivery	Stores delivery information connected to orders.

Table Structure

Describe Customers Table

```
desc Customers ;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Con

Field	Type	Null	Key	Default	Extra
CustomerID	int	NO	PRI	NULL	
FirstName	varchar(50)	YES		NULL	
LastName	varchar(50)	YES		NULL	
location	varchar(100)	YES		NULL	
Email	varchar(100)	YES		NULL	
phone	varchar(20)	YES		NULL	
AccountCreationDate	date	YES		NULL	
DateofBirth	date	YES		NULL	

Describe Categories Table

```
desc categories;
```

Result Grid | Filter Rows: | Export: | Wrap C

Field	Type	Null	Key	Default	Extra
CategoryID	int	NO	PRI	NULL	
CategoryName	varchar(50)	NO		NULL	

Describe Products Table

```
Desc Products;
```

Result Grid | Filter Rows: | Export: | Wrap

Field	Type	Null	Key	Default	Extra
OrderID	int	NO	PRI	NULL	
CustomerID	int	YES	MUL	NULL	
ProductID	int	YES	MUL	NULL	
OrderDate	date	YES		NULL	
Quantity	int	YES		NULL	
TotalAmount	decimal(10,2)	YES		NULL	
OrderStatus	varchar(30)	YES		NULL	

Describe Orders Table

```
Desc orders;
```

Result Grid | Filter Rows: | Export: | Wrap

Field	Type	Null	Key	Default	Extra
OrderID	int	NO	PRI	NULL	
CustomerID	int	YES	MUL	NULL	
ProductID	int	YES	MUL	NULL	
OrderDate	date	YES		NULL	
Quantity	int	YES		NULL	
TotalAmount	decimal(10,2)	YES		NULL	
OrderStatus	varchar(30)	YES		NULL	

Describe Payments Table

```
Desc payments;
```

Result Grid | Filter Rows: | Export: | Wrap Cell

Field	Type	Null	Key	Default	Extra
PaymentID	int	NO	PRI	NULL	
OrderID	int	YES	MUL	NULL	
PaymentDate	date	YES		NULL	
PaymentMethod	varchar(30)	YES		NULL	
Amount	decimal(10,2)	YES		NULL	
PaymentStatus	varchar(30)	YES		NULL	

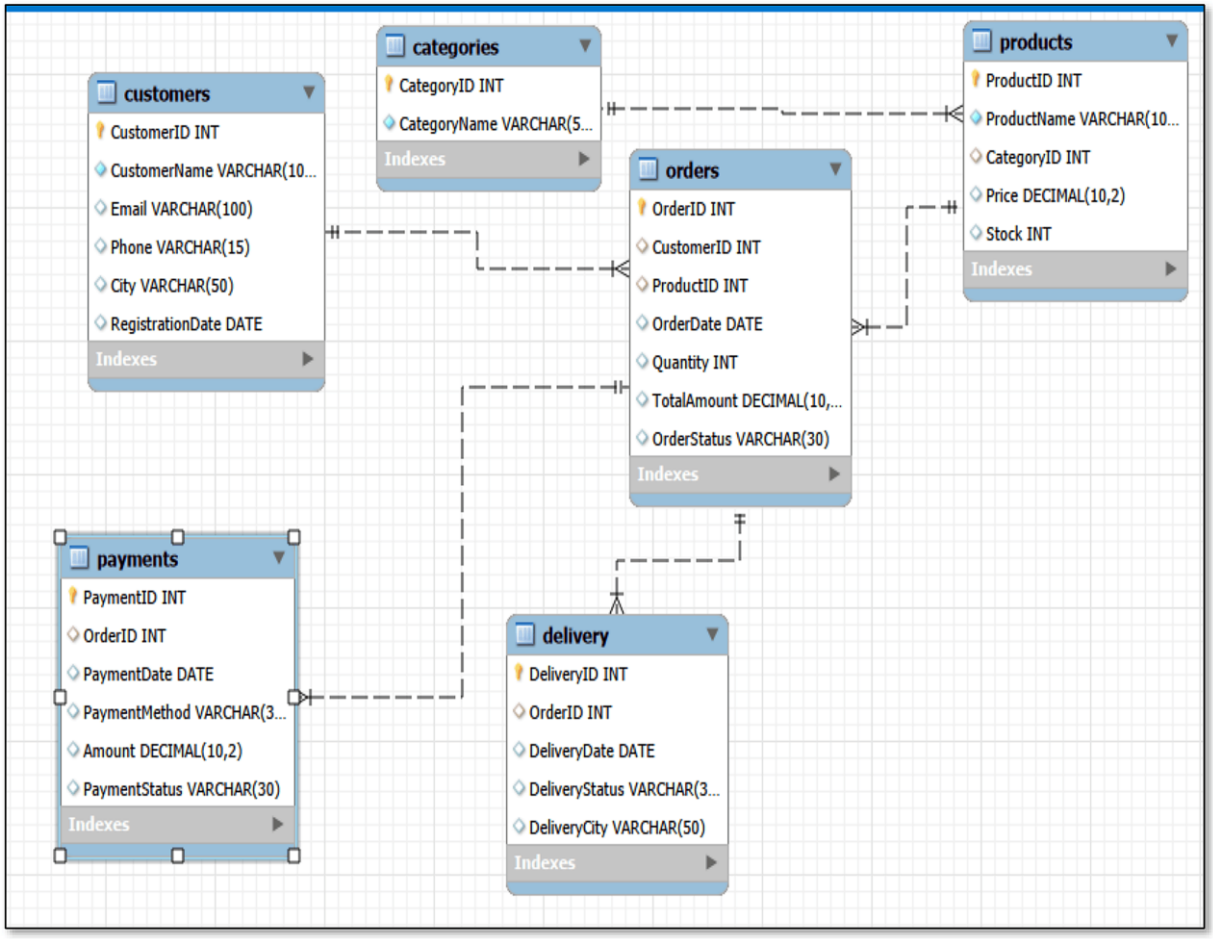
Describe Delivery Table

```
Desc delivery;
```

Result Grid | Filter Rows: | Export: | Wrap Cel

Field	Type	Null	Key	Default	Extra
PaymentID	int	NO	PRI	NULL	
OrderID	int	YES	MUL	NULL	
PaymentDate	date	YES		NULL	
PaymentMethod	varchar(30)	YES		NULL	
Amount	decimal(10,2)	YES		NULL	
PaymentStatus	varchar(30)	YES		NULL	

ER Diagram


customers
CustomerID INT
CustomerName VARCHAR(10...
Email VARCHAR(100)
Phone VARCHAR(15)
City VARCHAR(50)
RegistrationDate DATE
Indexes
categories
CategoryID INT
CategoryName VARCHAR(5...
Indexes
products
ProductID INT
ProductName VARCHAR(10...
CategoryID INT
Price DECIMAL(10,2)
Stock INT
Indexes
orders
OrderID INT
CustomerID INT
ProductID INT
OrderDate DATE
Quantity INT
TotalAmount DECIMAL(10,...
OrderStatus VARCHAR(30)
Indexes
payments
PaymentID INT
OrderID INT
PaymentDate DATE
PaymentMethod VARCHAR(3...
Amount DECIMAL(10,2)
PaymentStatus VARCHAR(30)
Indexes
delivery
DeliveryID INT
OrderID INT
DeliveryDate DATE
DeliveryStatus VARCHAR(3...
DeliveryCity VARCHAR(50)
Indexes

SQL Queries

Database Creation

```
CREATE DATABASE EcommerceDB;
USE EcommerceDB;
```

Table Creation

1. Customers

```
CREATE TABLE Customers (
  CustomerID INT PRIMARY KEY,
  CustomerName VARCHAR(100) NOT NULL,
  Email VARCHAR(100) UNIQUE,
  Phone VARCHAR(15),
  City VARCHAR(50),
  RegistrationDate DATE);
```

2. Categories

```
CREATE TABLE Categories (
  CategoryID INT PRIMARY KEY,
  CategoryName VARCHAR(50) NOT NULL);
```

3. Products

```
CREATE TABLE Products (
  ProductID INT PRIMARY KEY,
  ProductName VARCHAR(100) NOT NULL,
  CategoryID INT,
  Price DECIMAL(10,2),
  Stock INT,
  FOREIGN KEY (CategoryID)
  REFERENCES Categories(CategoryID);
```

4. Orders

```
CREATE TABLE Orders (
  OrderID INT PRIMARY KEY,
  CustomerID INT,
  ProductID INT,
  OrderDate DATE,
  Quantity INT,
  TotalAmount DECIMAL(10,2),
  OrderStatus VARCHAR(30),
  FOREIGN KEY (CustomerID) REFERENCE
  Customers(CustomerID),
  FOREIGN KEY (ProductID) REFERENCES Products(ProductID) );
```

5. Payments

```
CREATE TABLE Delivery (
  DeliveryID INT PRIMARY KEY,
  OrderID INT,
  DeliveryDate DATE,
  DeliveryStatus VARCHAR(30),
  DeliveryCity VARCHAR(50),
  FOREIGN KEY (OrderID)
  REFERENCES Orders(OrderID));
```

6. Delivery

```
CREATE TABLE Delivery (
  DeliveryID INT PRIMARY KEY,
  OrderID INT,
  DeliveryDate DATE,
  DeliveryStatus VARCHAR(30),
  DeliveryCity VARCHAR(50),
  FOREIGN KEY (OrderID) REFERENCES Orders(OrderID)
);
```

Inserted Values

Customers Table

```
INSERT INTO Customers
(CustomerID, CustomerName, Email, Phone, City, RegistrationDate)
VALUES
  (1,'Rahul Sharma','rahul.sharma@gmail.com','9876500001','Mumbai','2025-01-05'),
  (2,'Priya Patil','priya.patil@gmail.com','9876500002','Pune','2025-01-08'),
  (3,'Amit Shah','amit.shah@gmail.com','9876500003','Mumbai','2025-01-12'),
  (4,'Sneha Joshi','sneha.joshi@gmail.com','9876500004','Nashik','2025-01-15'),
  (5,'Rohan Mehta','rohan.mehta@gmail.com','9876500005','Thane','2025-01-20'),
  (6,'Neha Kulkarni','neha.k@gmail.com','9876500006','Pune','2025-01-25'),
  (7,'Akash Verma','akash.verma@gmail.com','9876500007','Delhi','2025-02-01'),
  (8,'Pooja Singh','pooja.singh@gmail.com','9876500008','Mumbai','2025-02-04'),
  (9,'Vikas Gupta','vikas.gupta@gmail.com','9876500009','Bangalore','2025-02-10'),
  (10,'Anjali Desai','anjali.desai@gmail.com','9876500010','Ahmedabad','2025-02-15'),
  (11,'Karan Malhotra','karan.m@gmail.com','9876500011','Delhi','2025-02-20'),
  (12,'Simran Kaur','simran.kaur@gmail.com','9876500012','Chandigarh','2025-02-25'),
  (13,'Arjun Rao','arjun.rao@gmail.com','9876500013','Hyderabad','2025-03-01'),
  (14,'Meera Nair','meera.nair@gmail.com','9876500014','Kochi','2025-03-05'),
  (15,'Sahil Khan','sahil.khan@gmail.com','9876500015','Mumbai','2025-03-10'),
  (16,'Riya Kapoor','riya.kapoor@gmail.com','9876500016','Jaipur','2025-03-15'),
  (17,'Varun Iyer','varun.iyer@gmail.com','9876500017','Chennai','2025-03-20'),
  (18,'Nisha Jain','nisha.jain@gmail.com','9876500018','Indore','2025-03-25'),
  (19,'Manish Yadav','manish.y@gmail.com','9876500019','Lucknow','2025-04-01'),
  (20,'Kavita Rao','kavita.rao@gmail.com','9876500020','Hyderabad','2025-04-05'),
  (21,'Deepak Kumar','deepak.k@gmail.com','9876500021','Patna','2025-04-10'),
  (22,'Ayesha Khan','ayesha.khan@gmail.com','9876500022','Mumbai','2025-04-15'),
  (23,'Mohit Agarwal','mohit.a@gmail.com','9876500023','Kolkata','2025-04-20'),
  (24,'Isha Patel','isha.patel@gmail.com','9876500024','Surat','2025-04-25'),
  (25,'Ravi Sinha','ravi.sinha@gmail.com','9876500025','Delhi','2025-05-01'),
  (26,'Tanya Roy','tanya.roy@gmail.com','9876500026','Kolkata','2025-05-05'),
  (27,'Nitin Joshi','nitin.joshi@gmail.com','9876500027','Pune','2025-05-10'),
  (28,'Shweta More','shweta.more@gmail.com','9876500028','Nashik','2025-05-15'),
  (29,'Aditya Mishra','aditya.m@gmail.com','9876500029','Lucknow','2025-05-20'),
  (30,'Komal Shah','komal.shah@gmail.com','9876500030','Ahmedabad','2025-05-25'),
  (31,'Yash Thakur','yash.thakur@gmail.com','9876500031','Shimla','2025-06-01'),
  (32,'Mansi Shah','mansi.shah@gmail.com','9876500032','Mumbai','2025-06-05'),
  (33,'Harsh Patel','harsh.patel@gmail.com','9876500033','Surat','2025-06-10'),
  (34,'Divya Menon','divya.menon@gmail.com','9876500034','Kochi','2025-06-15'),
  (35,'Raj Malhotra','raj.m@gmail.com','9876500035','Delhi','2025-06-20'),
  (36,'Pallavi Joshi','pallavi.j@gmail.com','9876500036','Pune','2025-06-25'),
  (37,'Suresh Reddy','suresh.r@gmail.com','9876500037','Hyderabad','2025-07-01'),
  (38,'Preeti Sharma','preeti.s@gmail.com','9876500038','Jaipur','2025-07-05'),
  (39,'Gaurav Singh','gaurav.s@gmail.com','9876500039','Delhi','2025-07-10'),
  (40,'Ritu Gupta','ritu.g@gmail.com','9876500040','Bangalore','2025-07-15'),
  (41,'Abhishek Jain','abhishek.j@gmail.com','9876500041','Indore','2025-07-20'),
  (42,'Sakshi Rao','sakshi.rao@gmail.com','9876500042','Chennai','2025-07-25'),
  (43,'Vivek Kumar','vivek.k@gmail.com','9876500043','Patna','2025-08-01'),
  (44,'Tanvi Desai','tanvi.d@gmail.com','9876500044','Mumbai','2025-08-05'),
  (45,'Kunal Mehta','kunal.m@gmail.com','9876500045','Thane','2025-08-10'),
  (46,'Aarav Singh','aarav.s@gmail.com','9876500046','Delhi','2025-08-15'),
```

```
  (47,'Muskan Verma','muskan.v@gmail.com','9876500047','Noida','2025-08-20'),
  (48,'Rakesh Patil','rakesh.p@gmail.com','9876500048','Pune','2025-08-25'),
  (49,'Ananya Iyer','ananya.i@gmail.com','9876500049','Chennai','2025-09-01'),
  (50,'Dev Kapoor','dev.k@gmail.com','9876500050','Mumbai','2025-09-05');
```

Categories Table

```
INSERT INTO Categories
(CategoryID, CategoryName)
VALUES
  (1, 'Electronics'),
  (2, 'Clothing'),
  (3, 'Footwear'),
  (4, 'Books'),
  (5, 'Beauty'),
  (6, 'Sports'),
  (7, 'Home & Kitchen'),
  (8, 'Grocery'),
  (9, 'Furniture'),
  (10, 'Mobile Accessories');
```

Products Table

```
INSERT INTO Products
(ProductID, ProductName, CategoryID, Price, Stock)
VALUES
  (1,'Laptop',1,55000,25),
  (2,'Smartphone',1,25000,40),
  (3,'Bluetooth Speaker',1,3200,55),
  (4,'Smart Watch',1,5000,35),
  (5,'T-Shirt',2,799,100),
  (6,'Jeans',2,1499,70),
  (7,'Formal Shirt',2,1799,60),
  (8,'Saree',2,2999,40),
  (9,'Running Shoes',3,2499,50),
  (10,'Sports Shoes',3,2999,45),
  (11,'Casual Shoes',3,1999,55),
  (12,'The Alchemist',4,399,60),
  (13,'Data Analytics Book',4,699,45),
  (14,'SQL Beginner Guide',4,599,50),
  (15,'Face Wash',5,299,90),
  (16,'Perfume',5,2499,55),
  (17,'Moisturizer',5,599,75),
  (18,'Cricket Bat',6,2200,25),
  (19,'Football',6,1299,50),
  (20,'Yoga Mat',6,999,40),
  (21,'Microwave Oven',7,8500,20),
  (22,'Mixer Grinder',7,3500,30),
  (23,'Electric Kettle',7,1800,35),
  (24,'Coffee Maker',7,4500,25),
  (25,'Rice',8,899,100),
  (26,'Cooking Oil',8,799,80),
  (27,'Tea Powder',8,399,90),
  (28,'Biscuits Pack',8,299,120),
  (29,'Office Chair',9,7500,15),
  (30,'Study Table',9,6500,20),
  (31,'Bookshelf',9,5500,18),
  (32,'Sofa Set',9,25000,10),
  (33,'Power Bank',10,1499,65),
  (34,'Wireless Charger',10,1999,45),
  (35,'USB Cable',10,499,100),
  (36,'Phone Case',10,399,120),
  (37,'Screen Protector',10,299,100),
  (38,'Laptop Sleeve',10,899,50),
  (39,'Gaming Headset',1,4500,30),
```

```
  (40,'Monitor',1,12000,25),
  (41,'Keyboard',1,2200,45),
  (42,'Gaming Mouse',1,1800,50),
  (43,'Hoodie',2,1999,55),
  (44,'Track Pants',2,1299,65),
  (45,'Sports Bag',6,1799,40),
  (46,'Protein Powder',6,2499,35),
  (47,'Hand Wash',5,199,100),
  (48,'Shampoo',5,499,100),
  (49,'Dining Table',9,15000,12),
  (50,'Air Fryer',7,6500,20);
```

Orders Table

```
INSERT INTO Orders
(OrderID, CustomerID, ProductID, OrderDate, Quantity, TotalAmount, OrderStatus)
VALUES
  (101,1,1,'2025-09-10',1,55000,'Delivered'),
  (102,2,3,'2025-09-11',2,6400,'Delivered'),
  (103,3,6,'2025-09-12',1,1499,'Delivered'),
  (104,4,21,'2025-09-13',1,8500,'Cancelled'),
  (105,5,4,'2025-09-14',1,5000,'Delivered'),
  (106,6,9,'2025-09-15',2,4998,'Shipped'),
  (107,7,13,'2025-09-16',2,1398,'Delivered'),
  (108,8,29,'2025-09-17',1,7500,'Delivered'),
  (109,9,18,'2025-09-18',1,2200,'Processing'),
  (110,10,22,'2025-09-19',1,3500,'Delivered'),
  (111,11,39,'2025-09-20',1,4500,'Delivered'),
  (112,12,33,'2025-09-21',2,2998,'Delivered'),
  (113,13,43,'2025-09-22',1,1999,'Shipped'),
  (114,14,40,'2025-09-23',1,12000,'Delivered'),
  (115,15,41,'2025-09-24',2,4400,'Delivered'),
  (116,16,14,'2025-09-25',1,599,'Delivered'),
  (117,17,30,'2025-09-26',1,6500,'Processing'),
  (118,18,23,'2025-09-27',2,3600,'Delivered'),
  (119,19,32,'2025-09-28',1,25000,'Delivered'),
  (120,20,50,'2025-09-29',1,6500,'Shipped'),
  (121,21,7,'2025-09-30',2,3598,'Delivered'),
  (122,22,8,'2025-10-01',1,2999,'Delivered'),
  (123,23,10,'2025-10-02',2,5998,'Delivered'),
  (124,24,16,'2025-10-03',1,2499,'Cancelled'),
  (125,25,25,'2025-10-04',3,2697,'Delivered'),
  (126,26,48,'2025-10-05',1,499,'Delivered'),
  (127,27,19,'2025-10-06',2,2598,'Shipped'),
  (128,28,35,'2025-10-07',1,499,'Delivered'),
  (129,29,27,'2025-10-08',2,798,'Delivered'),
  (130,30,28,'2025-10-09',5,1495,'Delivered'),
  (131,31,36,'2025-10-10',5,1995,'Delivered'),
  (132,32,37,'2025-10-11',1,299,'Processing'),
  (133,33,38,'2025-10-12',2,1798,'Delivered'),
  (134,34,24,'2025-10-13',1,4500,'Delivered'),
  (135,35,45,'2025-10-14',1,1799,'Shipped'),
  (136,36,46,'2025-10-15',1,2499,'Delivered'),
  (137,37,42,'2025-10-16',2,3600,'Delivered'),
  (138,38,47,'2025-10-17',2,398,'Delivered'),
  (139,39,17,'2025-10-18',3,1797,'Cancelled'),
  (140,40,26,'2025-10-19',2,1598,'Delivered'),
  (141,41,34,'2025-10-20',1,1999,'Processing'),
  (142,42,31,'2025-10-21',1,5500,'Delivered'),
  (143,43,44,'2025-10-22',2,2598,'Delivered'),
  (144,44,49,'2025-10-23',1,15000,'Shipped'),
  (145,45,20,'2025-10-24',3,2997,'Delivered'),
  (146,46,2,'2025-10-25',1,25000,'Delivered'),
  (147,47,5,'2025-10-26',3,2397,'Delivered'),
  (148,48,12,'2025-10-27',2,798,'Processing'),
```

```
  (149,49,15,'2025-10-28',1,299,'Delivered'),
  (150,50,50,'2025-10-29',2,13000,'Delivered');
```

Payments Table

```
INSERT INTO Payments
(PaymentID, OrderID, PaymentDate, PaymentMethod, Amount, PaymentStatus)
VALUES
  (1,101,'2025-09-10','UPI',55000,'Paid'),
  (2,102,'2025-09-11','Credit Card',6400,'Paid'),
  (3,103,'2025-09-12','Cash on Delivery',1499,'Paid'),
  (4,104,'2025-09-13','UPI',8500,'Refunded'),
  (5,105,'2025-09-14','Debit Card',5000,'Paid'),
  (6,106,'2025-09-15','UPI',4998,'Paid'),
  (7,107,'2025-09-16','Credit Card',1398,'Paid'),
  (8,108,'2025-09-17','UPI',7500,'Paid'),
  (9,109,'2025-09-18','Debit Card',2200,'Pending'),
  (10,110,'2025-09-19','UPI',3500,'Paid'),
  (11,111,'2025-09-20','Credit Card',4500,'Paid'),
  (12,112,'2025-09-21','UPI',2998,'Paid'),
  (13,113,'2025-09-22','Debit Card',1999,'Paid'),
  (14,114,'2025-09-23','Credit Card',12000,'Paid'),
  (15,115,'2025-09-24','UPI',4400,'Paid'),
  (16,116,'2025-09-25','Debit Card',599,'Paid'),
  (17,117,'2025-09-26','UPI',6500,'Pending'),
  (18,118,'2025-09-27','Credit Card',3600,'Paid'),
  (19,119,'2025-09-28','UPI',25000,'Paid'),
  (20,120,'2025-09-29','Debit Card',6500,'Paid'),
  (21,121,'2025-09-30','UPI',3598,'Paid'),
  (22,122,'2025-10-01','Credit Card',2999,'Paid'),
  (23,123,'2025-10-02','UPI',5998,'Paid'),
  (24,124,'2025-10-03','UPI',2499,'Refunded'),
  (25,125,'2025-10-04','Cash on Delivery',2697,'Paid'),
  (26,126,'2025-10-05','Debit Card',499,'Paid'),
  (27,127,'2025-10-06','UPI',2598,'Paid'),
  (28,128,'2025-10-07','Credit Card',499,'Paid'),
  (29,129,'2025-10-08','UPI',798,'Paid'),
  (30,130,'2025-10-09','Cash on Delivery',1495,'Paid'),
  (31,131,'2025-10-10','UPI',1995,'Paid'),
  (32,132,'2025-10-11','Debit Card',299,'Pending'),
  (33,133,'2025-10-12','Credit Card',1798,'Paid'),
  (34,134,'2025-10-13','UPI',4500,'Paid'),
  (35,135,'2025-10-14','Debit Card',1799,'Paid'),
  (36,136,'2025-10-15','Credit Card',2499,'Paid'),
  (37,137,'2025-10-16','UPI',3600,'Paid'),
  (38,138,'2025-10-17','Cash on Delivery',398,'Paid'),
  (39,139,'2025-10-18','UPI',1797,'Refunded'),
  (40,140,'2025-10-19','Debit Card',1598,'Paid'),
  (41,141,'2025-10-20','UPI',1999,'Pending'),
  (42,142,'2025-10-21','Credit Card',5500,'Paid'),
  (43,143,'2025-10-22','UPI',2598,'Paid'),
  (44,144,'2025-10-23','Debit Card',15000,'Paid'),
  (45,145,'2025-10-24','UPI',2997,'Paid'),
  (46,146,'2025-10-25','Credit Card',25000,'Paid'),
  (47,147,'2025-10-26','UPI',2397,'Paid'),
  (48,148,'2025-10-27','Debit Card',798,'Pending'),
  (49,149,'2025-10-28','UPI',299,'Paid'),
  (50,150,'2025-10-29','Credit Card',13000,'Paid');
```

Delivery Table

```
INSERT INTO Delivery
(DeliveryID, OrderID, DeliveryDate, DeliveryStatus, DeliveryCity)
VALUES
  (1,101,'2025-09-14','Delivered','Mumbai'),
```

```
(2,102,'2025-09-15','Delivered','Pune'),
(3,103,'2025-09-16','Delivered','Mumbai'),
(4,104,NULL,'Cancelled','Nashik'),
(5,105,'2025-09-18','Delivered','Thane'),
(6,106,NULL,'In Transit','Pune'),
(7,107,'2025-09-20','Delivered','Delhi'),
(8,108,'2025-09-21','Delivered','Mumbai'),
(9,109,NULL,'Processing','Bangalore'),
(10,110,'2025-09-23','Delivered','Ahmedabad'),
(11,111,'2025-09-24','Delivered','Delhi'),
(12,112,'2025-09-25','Delivered','Chandigarh'),
(13,113,NULL,'In Transit','Hyderabad'),
(14,114,'2025-09-28','Delivered','Kochi'),
(15,115,'2025-09-29','Delivered','Mumbai'),
(16,116,'2025-09-30','Delivered','Jaipur'),
(17,117,NULL,'Processing','Chennai'),
(18,118,'2025-10-01','Delivered','Indore'),
(19,119,'2025-10-02','Delivered','Lucknow'),
(20,120,NULL,'In Transit','Hyderabad'),
(21,121,'2025-10-04','Delivered','Patna'),
(22,122,'2025-10-05','Delivered','Mumbai'),
(23,123,'2025-10-06','Delivered','Kolkata'),
(24,124,NULL,'Cancelled','Surat'),
(25,125,'2025-10-08','Delivered','Delhi'),
(26,126,'2025-10-09','Delivered','Kolkata'),
(27,127,NULL,'In Transit','Pune'),
(28,128,'2025-10-11','Delivered','Nashik'),
(29,129,'2025-10-12','Delivered','Lucknow'),
(30,130,'2025-10-13','Delivered','Ahmedabad'),
(31,131,'2025-10-14','Delivered','Shimla'),
(32,132,NULL,'Processing','Mumbai'),
(33,133,'2025-10-16','Delivered','Surat'),
(34,134,'2025-10-17','Delivered','Kochi'),
(35,135,NULL,'In Transit','Delhi'),
(36,136,'2025-10-19','Delivered','Pune'),
(37,137,'2025-10-20','Delivered','Hyderabad'),
(38,138,'2025-10-21','Delivered','Jaipur'),
(39,139,NULL,'Cancelled','Delhi'),
(40,140,'2025-10-23','Delivered','Bangalore'),
(41,141,NULL,'Processing','Indore'),
(42,142,'2025-10-25','Delivered','Chennai'),
(43,143,'2025-10-26','Delivered','Patna'),
(44,144,NULL,'In Transit','Mumbai'),
(45,145,'2025-10-28','Delivered','Thane'),
(46,146,'2025-10-29','Delivered','Delhi'),
(47,147,'2025-10-30','Delivered','Noida'),
(48,148,NULL,'Processing','Pune'),
(49,149,'2025-10-31','Delivered','Chennai'),
(50,150,'2025-11-01','Delivered','Mumbai');
```

SQL Query Practice

Part A: 5 JOIN Questions

1. INNER JOIN

Question: Display customer names and their order details.

```
SELECT
  c.CustomerName,
  o.OrderID,
  o.OrderDate,
  o.TotalAmount
FROM Customers c
INNER JOIN Orders o
  ON c.CustomerID = o.CustomerID;
```

Output

CustomerName	OrderID	OrderDate	TotalAmount
Rahul Sharma	101	2025-09-10	55000.00
Priya Patil	102	2025-09-11	6400.00
Amit Shah	103	2025-09-12	1499.00
Sneha Joshi	104	2025-09-13	8500.00
Rohan Mehta	105	2025-09-14	5000.00
Neha Kulkarni	106	2025-09-15	4998.00
Akash Verma	107	2025-09-16	1398.00
Pooja Singh	108	2025-09-17	7500.00
Vikas Gupta	109	2025-09-18	2200.00
Anjali Desai	110	2025-09-19	3500.00
Karan Malhotra	111	2025-09-20	4500.00
Simran Kaur	112	2025-09-21	2998.00
Arjun Rao	113	2025-09-22	1999.00
Meera Nair	114	2025-09-23	12000.00
Sahil Khan	115	2025-09-24	4400.00
Riya Kapoor	116	2025-09-25	599.00
Varun Iyer	117	2025-09-26	6500.00

2. LEFT JOIN

Question: Display all customers, including customers who have never placed an order.

```
SELECT
  c.CustomerID,
  c.CustomerName,
  o.OrderID,
  o.TotalAmount
FROM Customers c
LEFT JOIN Orders o
  ON c.CustomerID = o.CustomerID;
```

Output

CustomerID	CustomerName	OrderID	TotalAmount
1	Rahul Sharma	101	55000.00
2	Priya Patil	102	6400.00
3	Amit Shah	103	1499.00
4	Sneha Joshi	104	8500.00
5	Rohan Mehta	105	5000.00
6	Neha Kulkarni	106	4998.00
7	Akash Verma	107	1398.00
8	Pooja Singh	108	7500.00
9	Vikas Gupta	109	2200.00
10	Anjali Desai	110	3500.00
11	Karan Malhotra	111	4500.00
12	Simran Kaur	112	2998.00

3. RIGHT JOIN

Question: Display all orders and the customer names associated with them.

```
SELECT
  c.CustomerName,
  o.OrderID,
  o.OrderDate,
  o.TotalAmount
FROM Customers c
RIGHT JOIN Orders o
  ON c.CustomerID = o.CustomerID;
```

Output

CustomerName	OrderID	OrderDate	TotalAmount
Rahul Sharma	101	2025-09-10	55000.00
Priya Patil	102	2025-09-11	6400.00
Amit Shah	103	2025-09-12	1499.00
Sneha Joshi	104	2025-09-13	8500.00
Rohan Mehta	105	2025-09-14	5000.00
Neha Kulkarni	106	2025-09-15	4998.00
Akash Verma	107	2025-09-16	1398.00
Pooja Singh	108	2025-09-17	7500.00
Vikas Gupta	109	2025-09-18	2200.00
Anjali Desai	110	2025-09-19	3500.00
Karan Malhotra	111	2025-09-20	4500.00
Simran Kaur	112	2025-09-21	2998.00
Arjun Rao	113	2025-09-22	1999.00
Meera Nair	114	2025-09-23	12000.00
Sahil Khan	115	2025-09-24	4400.00
Riya Kapoor	116	2025-09-25	599.00

4. CROSS JOIN

Question: Display every possible combination of customers and products.

```
SELECT
  c.CustomerName,
  p.ProductName
FROM Customers c
CROSS JOIN Products p;
```

Output

CustomerName	ProductName
Meera Kapoor	Laptop
Aarav Sharma	Laptop
Dev Kapoor	Laptop
Ananya Iyer	Laptop
Rakesh Patil	Laptop
Muskan Verma	Laptop
Aarav Singh	Laptop
Kunal Mehta	Laptop
Tanvi Desai	Laptop
Vivek Kumar	Laptop
Sakshi Rao	Laptop
Abhishek Jain	Laptop
Ritu Gupta	Laptop
Gaurav Singh	Laptop
Preeti Sharma	Laptop
Suresh Reddy	Laptop
Pallavi Joshi	Laptop
Raj Malhotra	Laptop
Divya Menon	Laptop
Harsh Patel	Laptop

5. SELF JOIN

Question: Display pairs of customers who live in the same city.

```
SELECT
  c1.CustomerName AS Customer1,
  c2.CustomerName AS Customer2,
  c1.City
FROM Customers c1
INNER JOIN Customers c2
  ON c1.City = c2.City
  AND c1.CustomerID < c2.CustomerID;
```

Output

Customer1	Customer2	City
Rahul Sharma	Amit Shah	Mumbai
Rahul Sharma	Pooja Singh	Mumbai
Rahul Sharma	Sahil Khan	Mumbai
Rahul Sharma	Ayesha Khan	Mumbai
Rahul Sharma	Mansi Shah	Mumbai
Rahul Sharma	Tanvi Desai	Mumbai
Rahul Sharma	Dev Kapoor	Mumbai
Rahul Sharma	Aarav Sharma	Mumbai
Priya Patil	Neha Kulkarni	Pune
Priya Patil	Nitin Joshi	Pune
Priya Patil	Pallavi Joshi	Pune
Priya Patil	Rakesh Patil	Pune
Amit Shah	Pooja Singh	Mumbai
Amit Shah	Sahil Khan	Mumbai
Amit Shah	Ayesha Khan	Mumbai
Amit Shah	Mansi Shah	Mumbai
Amit Shah	Tanvi Desai	Mumbai
Amit Shah	Dev Kapoor	Mumbai
Amit Shah	Aarav Sharma	Mumbai
Sneha Joshi	Shweta More	Nashik

Part B: 5 SUBQUERY Questions

Query 6

Question: Find products whose price is greater than the average product price.

```
SELECT *
FROM Products
WHERE Price > (
  SELECT AVG(Price)
  FROM Products
);
```

Output

ProductID	ProductName	CategoryID	Price	Stock
1	Laptop	1	55000.00	25
2	Smartphone	1	25000.00	40
4	Smart Watch	1	5000.00	35
21	Microwave Oven	7	8500.00	20
29	Office Chair	9	7500.00	15
30	Study Table	9	6500.00	20
31	Bookshelf	9	5500.00	18
32	Sofa Set	9	25000.00	10
40	Monitor	1	12000.00	25
49	Dining Table	9	15000.00	12
50	Air Fryer	7	6500.00	20
NULL	NULL	NULL	NULL	NULL

Query 7

Question: Find customers who have placed at least one order.

```
SELECT *
FROM Customers
WHERE CustomerID IN (
  SELECT CustomerID
  FROM Orders
);
```

Output

CustomerID	CustomerName	Email	Phone	City	RegistrationDate
1	Rahul Sharma	rahul.sharma@gmail.com	9876500001	Mumbai	2025-01-05
2	Priya Patil	priya.patil@gmail.com	9876500002	Pune	2025-01-08
3	Amit Shah	amit.shah@gmail.com	9876500003	Mumbai	2025-01-12
4	Sneha Joshi	sneha.joshi@gmail.com	9876500004	Nashik	2025-01-15
5	Rohan Mehta	rohan.mehta@gmail.com	9876500005	Thane	2025-01-20
6	Neha Kulkarni	neha.k@gmail.com	9876500006	Pune	2025-01-25
7	Akash Verma	akash.verma@gmail.com	9876500007	Delhi	2025-02-01
8	Pooja Singh	pooja.singh@gmail.com	9876500008	Mumbai	2025-02-04
9	Vikas Gupta	vikas.gupta@gmail.com	9876500009	Bangalore	2025-02-10
10	Anjali Desai	anjali.desai@gmail.com	9876500010	Ahmedabad	2025-02-15
11	Karan Malhotra	karan.m@gmail.com	9876500011	Delhi	2025-02-20
12	Simran Kaur	simran.kaur@gmail.com	9876500012	Chandigarh	2025-02-25
13	Arjun Rao	arjun.rao@gmail.com	9876500013	Hyderabad	2025-03-01
14	Meera Nair	meera.nair@gmail.com	9876500014	Kochi	2025-03-05
15	Sahil Khan	sahil.khan@gmail.com	9876500015	Mumbai	2025-03-10
16	Riya Kapoor	riya.kapoor@gmail.com	9876500016	Jaipur	2025-03-15
17	Varun Iyer	varun.iyer@gmail.com	9876500017	Chennai	2025-03-20
18	Nisha Jain	nisha.jain@gmail.com	9876500018	Indore	2025-03-25
19	Manish Yadav	manish.y@gmail.com	9876500019	Lucknow	2025-04-01
20	Kavita Rao	kavita.rao@gmail.com	9876500020	Hyderabad	2025-04-05

Query 8

Question: Find the most expensive product.

```
SELECT *
FROM Products
WHERE Price = (
  SELECT MAX(Price)
  FROM Products
);
```

Output

ProductID	ProductName	CategoryID	Price	Stock
1	Laptop	1	55000.00	25
NULL	NULL	NULL	NULL	NULL

Query 9

Question: Find orders having an amount greater than the average order amount.

```
SELECT *
FROM Orders
WHERE TotalAmount > (
  SELECT AVG(TotalAmount)
  FROM Orders
);
```

Output

OrderID	CustomerID	ProductID	OrderDate	Quantity	TotalAmount	OrderStatus
101	1	1	2025-09-10	1	55000.00	Delivered
102	2	3	2025-09-11	2	6400.00	Delivered
104	4	21	2025-09-13	1	8500.00	Cancelled
108	8	29	2025-09-17	1	7500.00	Delivered
114	14	40	2025-09-23	1	12000.00	Delivered
117	17	30	2025-09-26	1	6500.00	Processing
119	19	32	2025-09-28	1	25000.00	Delivered
120	20	50	2025-09-29	1	6500.00	Shipped
123	23	10	2025-10-02	2	5998.00	Delivered
144	44	49	2025-10-23	1	15000.00	Shipped
146	46	2	2025-10-25	1	25000.00	Delivered
150	50	50	2025-10-29	2	13000.00	Delivered
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Query 10

Question: Find customers who have never placed an order.

```
SELECT *
FROM Customers
WHERE CustomerID NOT IN (
  SELECT CustomerID
  FROM Orders
);
```

Output

CustomerID	CustomerName	Email	Phone	City	RegistrationDate
51	Aarav Sharma	aarav.sharma@gmail.com	9876543210	Mumbai	NULL
52	Meera Kapoor	meera.kapoor@gmail.com	9876501234	Delhi	NULL
NULL	NULL	NULL	NULL	NULL	NULL

Part C: 5 AGGREGATE FUNCTION Questions

Query 11

Question: Find the total sales amount.

```
SELECT SUM(TotalAmount) AS TotalSales
FROM Orders;
```

Output

TotalSales
277642.00

Query 12

Question: Find the average product price.

```
SELECT AVG(Price) AS AveragePrice
FROM Products;
```

Output

AveragePrice
4535.903846

Query 13

Question: Count the total number of customers.

```
SELECT COUNT(*) AS TotalCustomers
FROM Customers;
```

Output

TotalCustomers
52

Query 14

Question: Find the highest order amount.

```
SELECT MAX(TotalAmount) AS HighestOrder
FROM Orders;
```

Output

HighestOrder
55000.00

Query 15

Question: Find the total quantity of products sold.

```
SELECT SUM(Quantity) AS TotalProductsSold
FROM Orders;
```

Output

TotalProductsSold
83

Part D: 5 GROUP BY and HAVING Questions

Query 16

Question: Find the number of orders placed by each customer.

```
SELECT
  CustomerID,
  COUNT(*) AS TotalOrders
FROM Orders
GROUP BY CustomerID;
```

Output

CustomerID	TotalOrders
1	1
2	1
3	1
4	1
5	1
6	1
7	1
8	1
9	1
10	1
11	1
12	1
13	1

Query 17

Question: Find total sales generated by each customer.

```
SELECT
  CustomerID,
  SUM(TotalAmount) AS CustomerSales
FROM Orders
GROUP BY CustomerID;
```

Output

CustomerID	CustomerSales
1	55000.00
2	6400.00
3	1499.00
4	8500.00
5	5000.00
6	4998.00
7	1398.00
8	7500.00
9	2200.00
10	3500.00
11	4500.00
12	2998.00
13	1999.00

Query 18

Question: Find the number of products in each category.

```
SELECT
  CategoryID,
  COUNT(*) AS TotalProducts
FROM Products
GROUP BY CategoryID;
```

Output

CategoryID	TotalProducts
1	8
2	6
3	4
4	4
5	5
6	5
7	5
8	4
9	5
10	6

Query 19

Question: Find categories whose average product price exceeds ₹1,000. Display the category ID, average price, and highest product price for qualifying categories.

```
SELECT
  CategoryID,
  AVG(Price) AS AveragePrice,
  MAX(Price) AS HighestPrice
FROM Products
GROUP BY CategoryID
HAVING AVG(Price) > 1000;
```

Output

CategoryID	AveragePrice	HighestPrice
1	13587.500000	55000.00
2	1732.333333	2999.00
3	2099.000000	2999.00
6	1759.200000	2499.00
7	4960.000000	8500.00
9	11900.000000	25000.00

Query 20

Question: Display categories having more than five products.

```
SELECT
  CategoryID,
  COUNT(*) AS TotalProducts
FROM Products
GROUP BY CategoryID
HAVING COUNT(*) > 5;
```

Output

CategoryID	TotalProducts
1	8
2	6
10	6

Conclusion

The E-Commerce Management System using SQL project helped me understand how to design and manage a relational database. The project consists of six tables: Customers, Categories, Products, Orders, Payments, and Delivery, which store and organize important e-commerce information. Using SQL queries, I practiced different types of joins, subqueries, aggregate functions, GROUP BY, and HAVING clauses to retrieve and analyze data.

This project improved my knowledge of database management, data relationships, and SQL query writing. Overall, it provided practical experience in using SQL to manage e-commerce data efficiently and strengthened my foundation for future projects in data analytics.